# ShiftAddQwen mat-vec: symbols and the sign-select identity

ShiftAddQwen notes

October 2026

## 1 Setting

A linear layer computes $o = Wx$. The ShiftAdd format (ggml types `sa{2,...,6}[_128|_256]`) stores each row of $W$ as a sum of *sign planes* with one scale per plane and per group of weights along the input dimension:

$$w_j \;\approx\; \sum_{p=1}^{Q} \alpha_p \, b_{p,j}, \qquad b_{p,j} \in \{-1, +1\}. \tag{1}$$

The decode mat-vec in the patched `llama.cpp` evaluates one output row against one input block of this form. There are no lookup tables; the weights only ever appear as bit masks.

## 2 Symbols

| symbol | meaning |
|---|---|
| $j = 1, \ldots, G$ | index of a weight inside one *group* along the input dimension; $G \in \{64, 128, 256\}$ weights share one scale per plane |
| $w_j$ | the original (bf16) weight at position $j$ of the output row |
| $p = 1, \ldots, Q$ | index of the sign plane; $Q$ planes = $Q$ bits per weight ($Q = 2$ to 6 in the released models) |
| $b_{p,j} \in \{-1, +1\}$ | the sign of weight $j$ in plane $p$ |
| $s_j$ | shorthand for $b_{p,j}$ when a single plane $p$ is fixed (as in the kernel comment) |
| $\mathrm{bit}_{p,j} \in \{0, 1\}$ | how the sign is stored: bit $= 1 \Leftrightarrow s = +1$, bit $= 0 \Leftrightarrow s = -1$, i.e. $s_j = 2\,\mathrm{bit}_{p,j} - 1$ |
| $\alpha_p$ | scale of plane $p$ for this group: $\alpha_p = d \cdot a_p$ with one fp16 $d$ per 256-weight super-block and one u8 code $a_p$ per plane and group |
| $x_j$ | the input activation at position $j$ (fp16 in the model) |
| $y_j$ | the activation after llama.cpp's q8 activation quantization: an int8 value, with one scale $\delta$ per block of 32 activations, $x_j \approx \delta\, y_j$ |

## 3 The identity

The contribution of plane $p$ of one group to the output row is $\alpha_p \sum_j s_j\, y_j$. Substituting $s_j = 2\,\mathrm{bit}_j - 1$:

$$\sum_{j=1}^{G} s_j\, y_j \;=\; \sum_{j=1}^{G} (2\,\mathrm{bit}_j - 1)\, y_j \;=\; 2 \sum_{j:\,\mathrm{bit}_j=1} y_j \;-\; \sum_{j=1}^{G} y_j. \tag{2}$$

Both terms are evaluated with `dp4a`, a single CUDA instruction (available since compute capability 6.1, so on the V100 and A100): given two 32-bit registers $a$ and $b$ that each pack four int8 values, and a 32-bit accumulator $c$, it computes

$$\texttt{dp4a}(a, b, c) \;=\; c + a_0 b_0 + a_1 b_1 + a_2 b_2 + a_3 b_3 \tag{3}$$

in one cycle on the integer units. llama.cpp's own k-quant mat-vecs are built on it. In the ShiftAdd kernel $b$ holds four activations $y_j$ and $a$ holds four bytes that are each 0 or 1 (one nibble of sign bits expanded to bytes), so the "multiply" is a select: a 1 byte passes the activation, a 0 byte drops it. With $a =$ `0x01010101` it is a plain four-way sum. The two terms of Eq. (2) then have different costs:

- $\sum_j y_j$ does not depend on the plane or on the output row. It is computed once per activation block and reused for every plane and every row (`sx` in the kernel, one `dp4a` against `0x01010101`).
- $\sum_{\mathrm{bit}_j=1} y_j$ is a sum of activations *selected* by the bit mask. Four sign bits (one nibble) are expanded into four 0/1 bytes and one `dp4a` against the four int8 activations accumulates exactly the masked sum. No multiplication by a weight value ever happens.

## 4 Full output

For one output row, summing over groups $g$ and planes $p$, and restoring the activation scale $\delta$,

$$o \;\approx\; \sum_{g} \sum_{p=1}^{Q} \alpha_{g,p} \Big( 2 \sum_{j \in g:\,\mathrm{bit}_{g,p,j}=1} y_j \;-\; \sum_{j \in g} y_j \Big) \cdot \delta. \tag{4}$$

The scale $\alpha_{g,p}$ is applied once per group and plane (an fp32 multiply-add on the integer partial sum), so the inner loop is purely integer: per plane and per four weights, one `dp4a` plus a few integer ops to form the byte mask. Prefill (batch $> 1$) takes a different path: the planes are dequantized to fp16 with Eq. (1) and the GEMM goes through cuBLAS.

## 5 Relation to ShiftAddLLM

ShiftAddQwen keeps the weight representation and the goal of ShiftAddLLM (NeurIPS 2024) and replaces the fitting pipeline and the kernels.

**Kept.**

- The binary-coding reparameterization of Eq. (1): every weight is a sum of signed planes with per-group scales, so a mat-vec needs no weight multiplications.
- The block-wise and column-wise scale layouts of the paper (the `sab<Q>_<bc>x<br>` ggml family covers both), and mixed bit allocation across layers.

**Replaced.**

- **Kernel.** ShiftAddLLM's kernels precompute, for every $k$ consecutive activations, the $2^k$ possible signed partial sums in a lookup table (LUT), index it with each plane's bit pattern, and handle the scales with shifts. ShiftAddQwen's decode kernel uses Eq. (2) instead: the shared term $\sum_j y_j$ removes the sign handling and a dp4a byte mask does the selection, so there is no LUT storage, no LUT build, and the scale is one fp32 multiply-add per group and plane. The arithmetic that remains is select, integer add and a per-group scale. Prefill dequantizes to fp16 and uses cuBLAS.
- **Fit.** ShiftAddLLM fits the planes in weight space with its reparameterization-aware objective. ShiftAddQwen fits each linear to the layer's output (GPTQ column sweep with act-order, exact Hessian-weighted scale refit), chooses the bits and group size per linear from measured end-to-end KLD sensitivities, and then distils every scale against the BF16 teacher (Section 6.2). The last two steps have no counterpart in the paper.
- **Target and integration.** The paper evaluated dense OPT / LLaMA models in PyTorch with custom CUDA. ShiftAddQwen targets Qwen3.5's hybrid Gated-DeltaNet / attention architecture with its MTP head, keeps the DeltaNet gates, convolutions, norms, embedding and LM head in llama.cpp's own formats, and ships the decoder weights as new ggml types in a patched llama.cpp, so the models run in a standard server with tool calling, thinking and speculative decoding.

In one line: the ShiftAddLLM weight format, refitted with an output-aware, distillation-based pipeline, and deployed through a LUT-free llama.cpp kernel.

# 6 Why it beats the k-quant baselines

## 6.1 Memory

Every decoder weight group is stored as $Q$ sign planes plus one u8 scale per plane and group and one fp16 per 256-weight super-block, i.e. about $Q + 0.3$ bits per weight for $G = 128$; with the per-linear plan the released models average 4.4 and 4.0 bits per decoder weight. Against the reference Q4_K_M deployment (5.87 GB) this saves 13% (4.4 bits, 5.10 GB) and 19% (4.0 bits, 4.76 GB) of file size and GPU-resident weight memory, so a 32K-context server fits in 6–8 GB of GPU memory. The 4.0-bit model is also smaller than Q3_K_M (4.83 GB) while having 1.7× lower KLD.

## 6.2 Fidelity: a better size–quality frontier

Table 1 of `docs/results.md` gives, on the mixed-domain text, KLD 0.030 for the 4.4-bit model (5.10 GB) against 0.036 for UD-Q3_K_XL (5.24 GB), and 0.038 for the 4.0-bit model (4.76 GB)

against 0.067 for Q3_K_M (4.83 GB): at every size from 3.9 to 5.1 GB the ShiftAdd model has lower KLD than the k-quant of equal or larger size. Four ingredients contribute:

1. **Output-aware fit** (`saq/gptq_bcq.py`, `saq/colwise.py`). Each linear is quantized by a GPTQ column sweep with act-order against the layer Hessian; the scales of each group are fitted on the error-compensated weights and, after the sweep, all scales are re-solved exactly for the Hessian-weighted output error with the codes frozen. k-quants fit each block of 32 weights to a uniform grid by weight error alone.

2. **Mixed precision chosen by measurement** (`saq/sens_kld.py`, `saq/sensitivity.py`). The end-to-end KLD increase of quantizing each linear alone at every (bits, group) setting is measured, and a knapsack selects the setting per linear for the target average, instead of one fixed recipe by tensor type for every layer.

3. **End-to-end scale distillation** (`saq/finetune.py`). With the codes frozen, every scale is trained as $\alpha_0(1+\theta)$ to match the BF16 teacher's top-64 log-probabilities on $768 \times 2048$ tokens, then snapped back to the u8 grid.

4. **A more expressive grid for the same bits.** A $Q$-plane group has $2^Q$ levels at $\sum_p \pm\alpha_p$, so their spacing is set by $Q$ independent scales and adapts to each group's weight distribution, with about 0.3 bits of scale overhead against 4.5 bits per weight for Q4_K's uniform levels.

### 6.3 Speed

- **Decode** (tg128 on the A100): 134.6 / 129.9 tok/s for the 4.0 / 4.4-bit models against 125.2 for Q4_K_M (+8% / +4%). Decode is memory-bound, so the smaller weights are read faster, and the mat-vec of Section 3 is multiplication-free: the weights enter only as dp4a byte masks, with one scale applied per group and plane, and no lookup tables.

- **Prefill** (pp512): 5367 / 5525 tok/s against 4227 (+27% / +31%). The batched path dequantizes the planes to fp16 with Eq. (1) and runs the GEMM on the tensor cores through cuBLAS.

- **MTP speculative decoding** keeps its acceptance rate (0.60–0.68) and reaches 171.5 / 183.3 tok/s with the draft head, against 158.2 for Q4_K_M.